ApiBook

Complete Technical Documentation

REST API Reference Platform

ASP.NET Core 8 | PostgreSQL | Clean Architecture | JWT + AES Security

Version	Framework	Database
1.0.0	.NET 8.0	PostgreSQL

Prepared by: Development Team
Date: April 2026

1. Project Overview

ApiBook is a production-ready RESTful Web API built on ASP.NET Core 8 using Clean Architecture principles. It provides a structured way to store and retrieve developer Platforms (e.g., Linux, Docker, Windows) and their associated CLI Commands (e.g., ls -la, docker ps).

1.1 Purpose & Goals

- Demonstrate enterprise-grade Clean Architecture in a real .NET 8 project
- Provide secure CRUD operations with dual-layer authentication (JWT + API Key)
- Show AES-256-CBC encrypted connection string management
- Implement structured logging with Serilog (Console + daily rolling file)
- Support PostgreSQL via Entity Framework Core with proper migrations

1.2 Technology Stack

Technology	Version	Purpose
ASP.NET Core	8.0	Web API framework & middleware pipeline
Entity Framework Core	8.0	ORM for database access
Npgsql EF Core Provider	8.x	PostgreSQL database driver
PostgreSQL	14+	Primary relational database
Serilog	3.x	Structured logging (Console + File)
Microsoft.IdentityModel	7.x	JWT token validation
System.Security.Cryptography	Built-in	AES-256-CBC connection string encryption
OpenAPI	Built-in	Auto-generated API documentation

2. Architecture

ApiBook follows Clean Architecture (also called Onion or Hexagonal Architecture). Dependencies always point inward — outer layers depend on inner layers, never the reverse.

2.1 Layer Dependency Diagram

Clean Architecture — Dependency Flow

```
                    ┌──────────────────────────────────┐
                    │          ApiBook.Api             │
                    │   (Entry Point / Host)           │
                    │   Program.cs + Middleware        │
                    └──────────────────────────────────┘
                                  │
                                  ▼ depends on
                    ┌──────────────────────────────────┐
                    │       ApiBook.Presentation       │
                    │   Controllers (Auth, Commands,   │
                    │               Platforms)         │
                    └──────────────────────────────────┘
                                  │
                                  ▼ depends on
        ┌──────────────────────────────────────────────────┐
        │              ApiBook.Application                 │
        │   Contracts (Interfaces) | Services | DTOs       │
        └──────────────────────────────────────────────────┘
                  │ depends on            ▲ implements
    ┌─────────────▼──────────────┐   ┌────┴──────────────────┐
    │       ApiBook.Domain       │   │  ApiBook.Infrastructure │
    │   Entities: Platform,      │   │  Repos | EF Core |      │
    │             Command        │   │  JWT | ApiKey | Postgres │
    └────────────────────────────┘   └────────────────────────┘


  Cross-Cutting Concerns:
  ┌──────────────────────┐   ┌──────────────────────────┐
```

```
    |     ApiBook.Logging      |   |      ApiBook.Security        |
    |  Serilog Extensions      |   |  AES-256 Connection String   |
    |  Console + File Sinks    |   |  Encryption / Decryption     |
    └──────────────────────────┘   └──────────────────────────────┘
```

2.2 Request Flow Diagram

Full Request / Response Flow

```
  CLIENT (Browser / Postman / Frontend App)
       |
       |  HTTP/HTTPS Request
       ▼
  ┌──────────────────────────────────────────────────────────┐
  |                   ApiBook.Api                            |
  |                                                          |
  |  1. SerilogRequestLogging  ──── logs method, path, ms    |
  |       |                                                  |
  |  2. HTTPS Redirection      ──── http → https             |
  |       |                                                  |
  |  3. UseAuthentication      ──── parse JWT Bearer token   |
  |       |                                                  |
  |  4. ApiKeyMiddleware       ──── check X-API-Key header   |
  |       |    (POST/PUT/PATCH/DELETE only, skip /auth/token) |
  |       |                                                  |
  |  5. UseAuthorization       ──── enforce [Authorize] attr |
  |       |                                                  |
  |  6. MapControllers         ──── route to controller action |
  └──────────────────────────────────────────────────────────┘
                        |
                        |
        ┌───────────────▼────────────────────┐
        |     ApiBook.Presentation           |
        |  Controller receives request       |
        |  Calls ICommandService / IPlatform |
        └────────────────────────────────────┘
                        |
                        |
        ┌───────────────▼────────────────────┐
        |     ApiBook.Application            |
        |  Service: validate, map DTO        |
        |  Calls ICommandRepository          |
        └────────────────────────────────────┘
                        |
                        |
        ┌───────────────▼────────────────────┐
        |     ApiBook.Infrastructure         |
        |  Repository: EF Core query         |
        |  AppDbContext → PostgreSQL         |
        └────────────────────────────────────┘
                        |
                        |
        ┌───────────────▼────────────────────┐
        |         PostgreSQL Database        |
        |    tables: platforms, commands     |
        └────────────────────────────────────┘
                        |
      Response bubbles back up each layer →  HTTP Response to CLIENT
```

3. Project Structure

Solution Folder Tree — 7 Projects

```
ApiBook/
├── ApiBook.sln
│
├── ApiBook.Api/                    # Entry point & host
│   ├── Middleware/
│   │   └── ApiKeyMiddleware.cs     # X-API-Key validation
│   ├── Properties/
│   │   └── launchSettings.json    # HTTP:5227 | HTTPS:7097
│   ├── appsettings.json           # Configuration
│   └── Program.cs                 # DI + middleware pipeline
│
├── ApiBook.Application/            # Business logic (no infra deps)
│   ├── Contracts/
│   │   ├── IApiKeyValidator.cs
│   │   ├── ICommandRepository.cs
│   │   ├── ICommandService.cs
│   │   ├── IJwtTokenService.cs
│   │   ├── IPlatformRepository.cs
│   │   └── IPlatformService.cs
│   ├── DTOs/
│   │   ├── AuthLoginRequestDto.cs
│   │   ├── AuthTokenResponseDto.cs
│   │   ├── CommandCreateDto.cs
│   │   ├── CommandReadDto.cs
│   │   ├── CommandUpdateDto.cs
│   │   ├── PlatformCreateDto.cs
│   │   ├── PlatformReadDto.cs
│   │   └── PlatformUpdateDto.cs
│   └── Services/
│       ├── CommandService.cs
│       └── PlatformService.cs
│
├── ApiBook.Domain/                 # Pure domain — no dependencies
│   └── Entities/
│       ├── Command.cs
│       └── Platform.cs
│
├── ApiBook.Infrastructure/         # EF Core, Repos, JWT, Encryption
│   ├── Persistence/
│   │   ├── AppDbContext.cs
│   │   └── AppDbContextFactory.cs
│   ├── Repositories/
│   │   ├── CommandRepository.cs
│   │   └── PlatformRepository.cs
│   ├── Security/
│   │   ├── ConfigurationApiKeyValidator.cs
│   │   └── JwtTokenService.cs
│   └── DependencyInjection.cs
│
├── ApiBook.Logging/                # Serilog cross-cutting concern
│   └── LoggingExtensions.cs
│
├── ApiBook.Presentation/           # MVC Controllers
│   └── Controllers/
│       ├── AuthController.cs
│       ├── CommandsController.cs
```

```
│       └── PlatformsController.cs
│
└── ApiBook.Security/               # AES-256 encryption utility
    └── EncryptedConnectionStringResolver.cs
```

4. Project Details

4.1 ApiBook.Domain

The innermost layer. Contains only plain C# entity classes with no external dependencies. All other layers reference Domain; Domain references nothing.

Entity	Properties	Notes
Platform	Id (PK), Name (max 100), Publisher (max 100), Commands[]	One-to-many owner of Commands
Command	Id (PK), HowTo (max 200), CommandLine (max 200), PlatformId (FK), Platform?	Belongs to one Platform

4.2 ApiBook.Application

Business logic layer. Defines all contracts (interfaces) that outer layers must implement, owns the DTOs, and contains the service classes that orchestrate repository calls.

Contracts — Interface Definitions

Interface	Methods	Implemented By
ICommandRepository	GetAllAsync, GetByPlatformIdAsync, GetByIdAsync, AddAsync, UpdateAsync, DeleteAsync	CommandRepository (Infrastructure)
IPlatformRepository	GetAllAsync, GetByIdAsync, AddAsync, UpdateAsync, DeleteAsync	PlatformRepository (Infrastructure)
ICommandService	GetAllAsync, GetByPlatformIdAsync, GetByIdAsync, CreateAsync, UpdateAsync, DeleteAsync	CommandService (Application)
IPlatformService	GetAllAsync, GetByIdAsync, CreateAsync, UpdateAsync, DeleteAsync	PlatformService (Application)
IJwtTokenService	GenerateToken(subject)	JwtTokenService (Infrastructure)

Interface	Methods	Implemented By
IApiKeyValidator	IsValid(apiKey)	ConfigurationApiKeyValidator (Infra)

DTOs — Data Transfer Objects (sealed records)

DTO	Properties	Used In
AuthLoginRequestDto	Username, Password	POST /api/auth/token (request)
AuthTokenResponseDto	AccessToken, ExpiresAtUtc	POST /api/auth/token (response)
CommandCreateDto	HowTo, CommandLine, PlatformId	POST /api/commands
CommandReadDto	Id, HowTo, CommandLine, PlatformId	GET responses
CommandUpdateDto	HowTo, CommandLine, PlatformId	PUT /api/commands/{id}
PlatformCreateDto	Name, Publisher	POST /api/platforms
PlatformReadDto	Id, Name, Publisher	GET responses
PlatformUpdateDto	Name, Publisher	PUT /api/platforms/{id}

4.3 ApiBook.Infrastructure

Implements all interfaces defined in Application. Handles database access via EF Core, JWT generation, and API key validation.

Component	File	Responsibility
AppDbContext	Persistence/AppDbContext.cs	EF Core DbContext; Fluent API config; cascade delete
AppDbContextFactory	Persistence/AppDbContextFactory.cs	Design-time factory for dotnet ef migrations
CommandRepository	Repositories/CommandRepository.cs	Async CRUD with AsNoTracking reads
PlatformRepository	Repositories/PlatformRepository.cs	Async CRUD with AsNoTracking reads
JwtTokenService	Security/JwtTokenService.cs	HMAC-SHA256 JWT generation with issuer/audience/expiry
ConfigurationApiKeyValidator	Security/ConfigurationApiKeyValidator.cs	Validates X-API-Key against appsettings

Component	File	Responsibility
DependencyInjection	DependencyInjection.cs	AddInfrastructureServices() extension method

4.4 ApiBook.Presentation

Thin controller layer. Receives HTTP requests, calls the appropriate service, and returns HTTP responses. No business logic lives here.

Controller	Route Prefix	Actions
AuthController	/api/auth	IssueToken (POST /token)
CommandsController	/api/commands	GetAll, GetById, GetByPlatformId, Create, Update, Delete
PlatformsController	/api/platforms	GetAll, GetById, Create, Update, Delete

4.5 ApiBook.Logging

Cross-cutting concern. Provides two extension methods used in Program.cs:

- builder.AddStructuredLogging() — Configures Serilog with Console + File sinks, reads from appsettings.json
- app.UseStructuredRequestLogging() — Adds Serilog HTTP request middleware

Log files are written to: logs/apibook-YYYYMMDD.log (daily rolling)

4.6 ApiBook.Security

AES-256-CBC encryption/decryption utility for database connection strings. Keeps secrets out of source control.

```
ApiBook.Security — AES-256 Connection String Encryption
  Encryption Flow:
  ──────────────────────────────────────────────────────
  Environment Variable: APIBOOK_CONN_ENCRYPTION_KEY
          │
          │  SHA256 Hash  →  32-byte AES-256 key
          │
          ▼
  RandomNumberGenerator.GetBytes(16)  →  16-byte IV
          │
          │  AES-256-CBC Encrypt(plaintext)
          │
          ▼
  payload = IV (16 bytes) + CipherText
          │
          │  Base64 encode
          ▼
```

```
appsettings.json:  "Postgres": "enc:<BASE64_PAYLOAD>"

Decryption Flow (at runtime):
──────────────────────────────────────────────────────────
Read "enc:<BASE64>" from configuration
       │  Strip "enc:" prefix
       │  Base64 decode → payload
       │  payload[0..15]  = IV
       │  payload[16..]   = CipherText
       │  AES-256-CBC Decrypt with same env key
       ▼
Plain connection string → passed to EF Core

Resolution Priority:
1st: ConnectionStrings:{name}Plain  (dev override, plain text)
2nd: enc:<value>  (AES decrypt)
3rd: plain string (use as-is)
```

5. Authentication & Security Flow

5.1 Dual-Layer Security Model

ApiBook uses two independent security mechanisms for write operations. Both must pass for a mutating request to reach a controller action.

Dual-Layer Auth Flow

```
ANY Mutating Request (POST / PUT / PATCH / DELETE)
──────────────────────────────────────────────────────────
      │
      ▼
┌──────────────────────────────────────┐
│       ApiKeyMiddleware               │
│  Read: X-API-Key header              │
│                                      │
│  Missing or invalid?                 │──▶ 401 Unauthorized
│  Valid? ──────────────────────────── │──▶ next()
└──────────────────────────────────────┘
                  │
      ▼
┌──────────────────────────────────────┐
│       JWT Bearer Authentication      │
│  Read: Authorization: Bearer <tok>   │
│                                      │
│  [Authorize] endpoints only          │
│  Token missing / expired / invalid?  │──▶ 401 Unauthorized
│  Token valid? ────────────────────── │──▶ Controller Action
└──────────────────────────────────────┘

Exception:  /api/auth/token  is [AllowAnonymous]
            ApiKeyMiddleware skips this path
            No JWT required to obtain a token
```

5.2 Token Issuance Flow

```
JWT Token Issuance Sequence
  Client                    AuthController          JwtTokenService
    |                            |                        |
    |  POST /api/auth/token      |                        |
    |  { username, password }    |                        |
    |──────────────────────────► |                        |
    |                            |                        |
    |              Read Security:AuthUser from appsettings|
    |                            |                        |
    |              username match? password match?        |
    |                            | NO ──► 401 Unauthorized|
    |                            | YES                    |
    |                            |─── GenerateToken() ──►|
    |                            |                        |
    |                            |    Build JWT:          |
    |                            |    Issuer, Audience    |
    |                            |    Sub claim           |
    |                            |    HMAC-SHA256 sign    |
    |                            |    Expiry = now+60min  |
    |                            |◄── signed JWT string ──|
    |                            |                        |
    |◄── 200 OK ────────────────|                        |
    |  { accessToken,            |                        |
    |    expiresAtUtc }          |                        |
```

5.3 JWT Token Structure

Field	Value / Source
Algorithm	HMAC-SHA256 (HS256)
Issuer	appsettings: Security:Jwt:Issuer (default: "ApiBook")
Audience	appsettings: Security:Jwt:Audience (default: "ApiBookClients")
Subject	Authenticated username
Expiry	appsettings: Security:Jwt:ExpiryMinutes (default: 60 minutes)
Clock Skew	30 seconds (validation tolerance)
Signing Key	Security:Jwt:SigningKey — must be 32+ characters

6. API Endpoints

Base URL (Development): https://localhost:7097

6.1 Authentication

Method	Endpoint	Auth	Description
POST	/api/auth/token	None (AllowAnonymous)	Issue a JWT bearer token

POST /api/auth/token

Request Body:

```
{
  "username": "admin",
  "password": "ChangeMeNow!"
}
```

Success Response (200 OK):

```
{
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "expiresAtUtc": "2026-04-29T14:00:00Z"
}
```

Error Responses:

Status Code	Condition
401 Unauthorized	Username or password is incorrect
500 Internal Server Error	Auth credentials not configured in appsettings

6.2 Platforms

Method	Endpoint	Auth Required	Description
GET	/api/platforms	No	Get all platforms
GET	/api/platforms/{id}	No	Get platform by ID
POST	/api/platforms	JWT + API Key	Create new platform
PUT	/api/platforms/{id}	JWT + API Key	Update existing platform
DELETE	/api/platforms/{id}	JWT + API Key	Delete platform (cascades commands)

Platform Create/Update body:

```
{ "name": "Linux", "publisher": "Linus Torvalds" }
```

Platform Read response:

```
{ "id": 1, "name": "Linux", "publisher": "Linus Torvalds" }
```

6.3 Commands

Method	Endpoint	Auth Required	Description
GET	/api/commands	No	Get all commands
GET	/api/commands/{id}	No	Get command by ID
GET	/api/commands/platform/{platformId}	No	Get commands by platform
POST	/api/commands	JWT + API Key	Create new command
PUT	/api/commands/{id}	JWT + API Key	Update existing command
DELETE	/api/commands/{id}	JWT + API Key	Delete command

Command Create/Update body:

```
{ "howTo": "List all files", "commandLine": "ls -la", "platformId": 1 }
```

Command Read response:

```
{ "id": 1, "howTo": "List all files", "commandLine": "ls -la", "platformId": 1 }
```

6.4 HTTP Status Code Reference

Status Code	Meaning	Returned When
200 OK	Success	Successful GET
201 Created	Resource created	Successful POST; includes Location header
204 No Content	Success no body	Successful PUT or DELETE
401 Unauthorized	Auth failed	Invalid/missing JWT or API Key
404 Not Found	Not found	Resource with given ID does not exist
500 Internal Server Error	Server error	Misconfiguration or unhandled exception

6.5 Request Headers for Write Operations

Header	Value	Required
Authorization	Bearer eyJhbGci...	Yes — JWT token from /api/auth/token
X-API-Key	your-configured-api-key	Yes — must match Security:ApiKey in config
Content-Type	application/json	Yes — for POST and PUT

7. Database Design

7.1 Entity Relationship Diagram

ER Diagram — platforms & commands

```
┌──────────────────────────────┐          ┌──────────────────────────────────────┐
│          platforms           │          │               commands               │
├──────────────────────────────┤          ├──────────────────────────────────────┤
│  id          SERIAL  PK      │◄─────────│  id            SERIAL  PK            │
│  name        VARCHAR(100) NN │   1    N │  how_to        VARCHAR(200) NN       │
│  publisher   VARCHAR(100) NN │          │  command_line  VARCHAR(200) NN       │
└──────────────────────────────┘          │  platform_id   INT FK → platforms.id │
                                           └──────────────────────────────────────┘

Cardinality:  Platform  1 ──────────────── N  Command
On Delete:    CASCADE — deleting a Platform deletes all its Commands
Constraint:   platform_id is NOT NULL (every command needs a platform)
```

7.2 Table Definitions

platforms table

```
CREATE TABLE platforms (
    id          SERIAL          PRIMARY KEY,
    name        VARCHAR(100)    NOT NULL,
    publisher   VARCHAR(100)    NOT NULL
);
```

commands table

```
CREATE TABLE commands (
```

```
    id            SERIAL          PRIMARY KEY,
    how_to        VARCHAR(200)    NOT NULL,
    command_line  VARCHAR(200)    NOT NULL,
    platform_id   INTEGER         NOT NULL
        REFERENCES platforms(id) ON DELETE CASCADE
);
```

7.3 EF Core Fluent Configuration

Entity	Table	Key Config
Platform	platforms	HasKey(Id), Name max 100 required, Publisher max 100 required
Command	commands	HasKey(Id), HowTo max 200, CommandLine max 200, HasOne(Platform).WithMany(Commands).OnDelete(Cascade)

8. Middleware Pipeline

The middleware pipeline in Program.cs is order-sensitive. Each middleware wraps the next, forming a chain. The order below is critical for correct behavior.

Middleware Pipeline — Execution Order

```
Incoming Request
      |
      ▼
┌──────────────────────────────────────────────────────────────┐
│  1. Serilog Request Logging                                  │
│     app.UseStructuredRequestLogging()                        │
│     Logs: method, path, status code, elapsed ms             │
└──────────────────────────────────────────────────────────────┘
                              |
                              ▼
┌──────────────────────────────────────────────────────────────┐
│  2. HTTPS Redirection                                        │
│     app.UseHttpsRedirection()                                │
│     http://... → https://...                                 │
└──────────────────────────────────────────────────────────────┘
                              |
                              ▼
┌──────────────────────────────────────────────────────────────┐
│  3. Authentication                                           │
│     app.UseAuthentication()                                  │
│     Parses Authorization: Bearer header                      │
│     Validates JWT signature, issuer, audience, expiry        │
│     Sets HttpContext.User principal                          │
└──────────────────────────────────────────────────────────────┘
                              |
                              ▼
┌──────────────────────────────────────────────────────────────┐
│  4. ApiKeyMiddleware (Custom)                                │
│     app.UseMiddleware<ApiKeyMiddleware>()                    |
```

```
    │     Only for POST / PUT / PATCH / DELETE                    │
    │     Skips /api/auth/token                                   │
    │     Reads X-API-Key header → validates via IApiKeyValidator │
    │     Invalid → 401 immediately, stops pipeline               │
    └─────────────────────────────────────────────────────────────┘
                              │
    ┌─────────────────────────▼───────────────────────────────────┐
    │  5. Authorization                                           │
    │     app.UseAuthorization()                                  │
    │     Enforces [Authorize] attribute on controller actions    │
    │     Anonymous actions pass through freely                   │
    └─────────────────────────────────────────────────────────────┘
                              │
    ┌─────────────────────────▼───────────────────────────────────┐
    │  6. Controller Routing                                      │
    │     app.MapControllers()                                    │
    │     Routes to matching controller/action                    │
    │     Returns HTTP response                                   │
    └─────────────────────────────────────────────────────────────┘
                              │
         ▼
   Outgoing Response (bubbles back up through the pipeline)
```

9. EF Core Migration Commands

Prerequisites
Install EF Core CLI tools once: dotnet tool install --global dotnet-ef Set environment variable: APIBOOK_CONN_ENCRYPTION_KEY=<your-key> (if using encrypted connection strings) All commands run from the solution root directory.

9.1 Create a New Migration

```
dotnet ef migrations add <MigrationName> \
  --project ApiBook.Infrastructure \
  --startup-project ApiBook.Api \
  --output-dir Persistence/Migrations

# Example:
dotnet ef migrations add InitialCreate \
  --project ApiBook.Infrastructure \
  --startup-project ApiBook.Api \
  --output-dir Persistence/Migrations
```

9.2 Apply Migrations (Update Database)

```
dotnet ef database update \
  --project ApiBook.Infrastructure \
  --startup-project ApiBook.Api

# Apply up to a specific migration:
dotnet ef database update InitialCreate \
  --project ApiBook.Infrastructure \
  --startup-project ApiBook.Api
```

9.3 Rollback / Remove

```
# Rollback to a previous migration (reverts DB):
dotnet ef database update <PreviousMigrationName> \
  --project ApiBook.Infrastructure \
  --startup-project ApiBook.Api

# Remove the last unapplied migration (file only):
dotnet ef migrations remove \
  --project ApiBook.Infrastructure \
  --startup-project ApiBook.Api
```

9.4 Other Useful Commands

```
# List all migrations and their applied status:
dotnet ef migrations list \
  --project ApiBook.Infrastructure \
  --startup-project ApiBook.Api

# Generate SQL script (without applying):
dotnet ef migrations script \
  --project ApiBook.Infrastructure \
  --startup-project ApiBook.Api \
  --output migration.sql

# Drop the entire database (DANGEROUS!):
dotnet ef database drop \
  --project ApiBook.Infrastructure \
  --startup-project ApiBook.Api
```

Important Note
AppDbContextFactory reads appsettings.json from the ApiBook.Api directory at design time. Make sure the APIBOOK_CONN_ENCRYPTION_KEY environment variable is set before running any EF commands if your connection string uses enc: prefix encryption.

10. Configuration Reference

10.1 appsettings.json

```
{
  "ConnectionStrings": {
    "Postgres": "enc:REPLACE_WITH_AES_ENCRYPTED_VALUE"
  },
  "Security": {
    "ApiKey": "REPLACE_WITH_STRONG_API_KEY",
    "AuthUser": {
      "Username": "admin",
      "Password": "ChangeMeNow!"
    },
    "Jwt": {
      "Issuer":       "ApiBook",
      "Audience":     "ApiBookClients",
      "SigningKey":   "REPLACE_WITH_LONG_RANDOM_SIGNING_KEY_32PLUS_CHARS",
      "ExpiryMinutes": 60
    }
  },
  "Serilog": {
    "MinimumLevel": {
      "Default": "Information",
      "Override": {
        "Microsoft": "Warning",
        "Microsoft.AspNetCore": "Warning"
      }
    }
  },
  "AllowedHosts": "*"
}
```

10.2 Configuration Keys Reference

Key	Required	Description
ConnectionStrings:Postgres	Yes	PostgreSQL connection string (plain or enc: prefixed)
Security:ApiKey	Yes	API key checked in X-API-Key header for write ops
Security:AuthUser:Username	Yes	Admin username for token issuance
Security:AuthUser:Password	Yes	Admin password for token issuance
Security:Jwt:Issuer	Yes	JWT issuer claim (e.g. ApiBook)
Security:Jwt:Audience	Yes	JWT audience claim (e.g. ApiBookClients)
Security:Jwt:SigningKey	Yes	HMAC-SHA256 signing key (min 32 chars)

Key	Required	Description
Security:Jwt:ExpiryMinutes	No	Token lifetime in minutes (default: 60)
APIBOOK_CONN_ENCRYPTION_KEY	Cond	Env var — required if connection string uses enc:

10.3 Environment Variables

Variable	Required	Purpose
APIBOOK_CONN_ENCRYPTION_KEY	If using enc:	AES-256 key for decrypting connection string
ASPNETCORE_ENVIRONMENT	Optional	Development / Staging / Production

11. Getting Started

11.1 Prerequisites

Requirement	Version	Link
.NET SDK	8.0+	https://dotnet.microsoft.com/download
PostgreSQL	14+	https://www.postgresql.org/download
EF Core CLI	Latest	dotnet tool install --global dotnet-ef

11.2 Setup Steps

1. **Clone the repository**

```
git clone https://github.com/yourusername/ApiBook.git
cd ApiBook
```

2. **Set the encryption key environment variable**

```
# Linux / macOS:
export APIBOOK_CONN_ENCRYPTION_KEY="your-secret-key"

# Windows PowerShell:
$env:APIBOOK_CONN_ENCRYPTION_KEY = "your-secret-key"

# Windows CMD:
```

```
set APIBOOK_CONN_ENCRYPTION_KEY=your-secret-key
```

3. **Update appsettings.json with your values**

- Set PostgreSQL connection string (plain or AES-encrypted)
- Set a strong ApiKey (use a GUID or random 32+ char string)
- Set AuthUser credentials (change from defaults!)
- Set Jwt:SigningKey (must be 32+ characters)

4. **Apply database migrations**

```
dotnet ef database update \
  --project ApiBook.Infrastructure \
  --startup-project ApiBook.Api
```

5. **Run the API**

```
dotnet run --project ApiBook.Api
```

API available at:

- HTTP: http://localhost:5227
- HTTPS: https://localhost:7097
- OpenAPI (dev only): https://localhost:7097/openapi/v1.json

11.3 Quick Test Sequence

Step 1 — Get a JWT Token

```
POST https://localhost:7097/api/auth/token
Content-Type: application/json

{ "username": "admin", "password": "ChangeMeNow!" }

# Copy the accessToken from the response
```

Step 2 — Create a Platform (uses token + API key)

```
POST https://localhost:7097/api/platforms
Authorization: Bearer <your_token>
X-API-Key: <your_api_key>
Content-Type: application/json

{ "name": "Linux", "publisher": "Linus Torvalds" }
```

Step 3 — Add a Command

```
POST https://localhost:7097/api/commands
Authorization: Bearer <your_token>
X-API-Key: <your_api_key>
Content-Type: application/json

{ "howTo": "List all files", "commandLine": "ls -la", "platformId": 1 }
```

Step 4 — Read Data (no auth needed)

```
GET https://localhost:7097/api/platforms
GET https://localhost:7097/api/commands
GET https://localhost:7097/api/commands/platform/1
```

12. Logging

ApiBook uses Serilog for structured logging. All log events include contextual properties (RequestId, UserAgent, etc.) automatically.

12.1 Log Sinks

Sink	Output	Rolling Strategy
Console	Terminal / stdout	N/A — real-time
File	logs/apibook-YYYYMMDD.log	Daily rolling (new file each day)

12.2 Log Levels

Namespace	Level	Reason
Default (application)	Information	Log all app-level events
Microsoft.*	Warning	Suppress noisy framework info logs
Microsoft.AspNetCore.*	Warning	Suppress request routing debug logs

12.3 What Gets Logged

- Every HTTP request: method, path, response code, elapsed milliseconds

- Application startup and shutdown events
- Service registration and DI resolution
- Any unhandled exceptions (with full stack trace)
- Custom log statements from services and controllers

12.4 Sample Log Output

```
[13:45:22 INF] Application starting up
[13:45:22 INF] Now listening on: https://localhost:7097
[13:45:30 INF] HTTP GET /api/platforms responded 200 in 12.4 ms
[13:45:35 INF] HTTP POST /api/commands responded 201 in 34.1 ms
[13:45:40 WRN] HTTP POST /api/platforms responded 401 in 2.3 ms
```

13. Design Patterns Used

Pattern	Where Used	Benefit
Clean Architecture	Entire solution (7 projects)	Separation of concerns, testability, maintainability
Repository Pattern	Infrastructure/Repositories	Abstracts EF Core; easy to swap data source
Dependency Injection	Program.cs + DependencyInjection.cs	Loose coupling, IoC, testability
DTO Pattern	Application/DTOs (sealed records)	Decouples API contract from domain model
Interface Segregation	Application/Contracts	Each interface has a single responsibility
Extension Method Pattern	Logging & Infrastructure	Clean, chainable builder configuration
Factory Pattern	AppDbContextFactory	Design-time EF Core context creation for migrations
Middleware Chain	ApiKeyMiddleware	Cross-cutting security concern without touching controllers
Decorator/Pipeline	ASP.NET middleware pipeline	Composable request processing

14. Security Checklist

Item	Status	Notes
JWT token validation	Implemented	Issuer, Audience, Signature, Lifetime, ClockSkew
API Key middleware	Implemented	Header-based, configurable, skips auth endpoint
AES-256-CBC connection string	Implemented	IV prepended, SHA256 key derivation from env var
Secrets out of source control	Implemented	Uses env vars + enc: prefix; never plain in appsettings
HTTPS enforcement	Implemented	UseHttpsRedirection() in pipeline
Cancellation token support	Implemented	All async operations support graceful cancellation
Input length limits	Implemented	EF Fluent API: Name/Publisher max 100, HowTo/Cmd max 200
Cascade delete safety	Implemented	Platform delete cascades to Commands automatically
Authorization on write ops	Implemented	[Authorize] + ApiKeyMiddleware dual-layer

ApiBook — Complete Technical Documentation

Built with Clean Architecture | .NET 8 | PostgreSQL | Serilog | JWT